SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

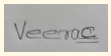
Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – SCENARIO BASED ASSIGNMENT

Course Code:	CSA17	**Course Title:**	Artificial Intelligence
CO Assessed:	CO2	**Assessment:**	Scenario Based Assignment
Weightage:	20% of CIA	**Duration / Marks:**	60 mins
CO2 Statement:	Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)		
Student Name:	C Veera Bharadwaj Kumar	**Reg. No.:**	192565061
Section / Batch:	CSE(CS)/2025	**Date:**	22-10-2026

Code of Conduct

I C Veera Bharadwaj Kumar(192565061) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: 

Instructions

- Read each scenario carefully before answering (Off-Class Assessment)
- Answer all five scenario-based questions. Each question carries and is designed to be answered in about 12 minutes.
- Identify the key issues in each scenario, apply the relevant AI technique, and justify every decision with clear reasoning.
- Questions are drawn from Informed Search (Greedy, A*), Local Search and Optimization, Online Search, Constraint Satisfaction Problems, and Adversarial Search (Minimax, Alpha-Beta, evaluation functions).
- Marks will be awarded using the Scenario Based Assignment rubric: understanding of scenario, identification of key issues, application of concepts, decision making and clarity of response.
- Upload your work in GCR and Github.

Section / Topic	Focus	Question Nos.
Informed Search Strategies	Greedy best-first search, A* search, heuristic functions	**Q1**
Local Search and Online Search	Local search algorithms and optimization problems (hill climbing, simulated annealing, genetic algorithms), online search agent	**Q2, Q3**

Constraint Satisfaction Problem	Variables, domains, constraints, backtracking search, constraint propagation (forward checking)	**Q4**
Game Playing: Adversarial Search	Games, minimax algorithm, alphabeta pruning, evaluation functions	**Q5**

Rubrics for Scenario Based Assignment

Criteria	Weightage	Level 4 – Exemplary (100% of marks)	Level 3 – Proficient (75%)	Level 2 – Developing (50%)	Level 1 – Beginning (25%)
Understanding of Scenario	**20%**	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	**20%**	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	**25%**	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	**20%**	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	**15%**	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Q. No. / Criteria	Understanding of Scenario	Identification of Key Issues	Application of Concepts	Decision Making	Clarity of Response	Sub. Total	Total %
1							
2							
3							
4							
5							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Annexure A – Scenario Based Assignment

Question 1: Emergency Medicine Delivery Drones in a Flood Zone

Topic: Informed search strategies (Greedy best-first search, A, heuristic functions) | 12 minutes*

A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic: roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to:

- A heuristic estimate (straight-line distance) to the destination.
- Partial real-time updates about blocked paths.
- Variable movement costs depending on terrain and weather.

Due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

(a) Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty.

(b) Explain how A* would handle the same situation, including how it balances cost and heuristic.

(c) Analyze the impact of heuristic accuracy on both algorithms.

(d) Discuss how dynamic changes (blocked paths) affect search performance.

(e) Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety.

Question 2: Smart City Traffic Signal Optimization

Topic: Local search algorithms and optimization problems (hill climbing, simulated annealing, genetic algorithms) | 12 minutes

A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize:

- Total waiting time.
- Fuel consumption. • Traffic congestion.

The system must continuously adjust signals based on traffic flow. However, the search space is extremely large, traffic patterns change dynamically, and the system may get stuck in locally optimal solutions.

Tasks:

(a) Formulate this as an optimization problem and explain why traditional search is inefficient.

(b) Explain how Hill Climbing would work in this scenario and identify its limitations.

(c) Discuss issues like local maxima, plateaus, and ridges with real-world interpretation.

(d) Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations.

(e) Recommend the best approach and justify your choice based on scalability and adaptability.

Question 3: Autonomous Rover Exploring Mars

Topic: Online search agent | 12 minutes

An autonomous rover is deployed on Mars to explore unknown terrain. It must:

- Navigate safely to collect samples.
- Avoid hazards (craters, rocks).
- Operate with limited sensing capability.
- Make decisions without a complete map.

The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

(a) Explain why this problem requires an online search agent instead of offline search.

(b) Analyze the challenges of partial observability and dynamic environments.

(c) Describe how the rover builds and updates its internal model.

(d) Compare online search with uninformed and informed search strategies.

(e) Justify the most suitable approach considering uncertainty, adaptability, and safety.

Question 4: University Exam Timetable Generation

Topic: Constraint satisfaction problem (backtracking search, constraint propagation, forward checking) | 12 minutes

A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints:

- No student should have overlapping exams.
- Limited number of exam halls.
- Certain subjects must be scheduled before others.
- Faculty availability constraints.
- Special accommodations for some students.

The complexity increases as the number of courses and students grows.

Tasks:

(a) Formulate the problem as a Constraint Satisfaction Problem (CSP): clearly define the variables, domains, and constraints with explanation.

(b) Explain how backtracking search works in this scenario.

(c) Discuss how constraint propagation (for example, forward checking) improves efficiency.

(d) Analyze real-world challenges and justify the best strategy for scalability.

Question 5: Strategic Game AI for Real-Time Decision Making

Topic: Game playing: adversarial search (minimax, alpha-beta pruning, evaluation functions) | 12 minutes

A gaming company is developing an AI opponent for a real-time strategy game. The AI must:

- Compete against human players.
- Make decisions within strict time limits.
- Evaluate complex game states with multiple possible moves.

The decision tree is extremely large, making computation expensive.

Tasks:

(a) Explain how the Minimax algorithm models this problem.

(b) Analyze the computational challenges of large game trees.

(c) Explain how Alpha-Beta pruning improves efficiency with detailed reasoning.

(d) Design an evaluation function and justify the factors considered.

(e) Recommend a strategy for balancing optimality and real-time performance.

Question 1: Emergency Medicine Delivery Drones in a Flood Zone

Topic: Informed Search – Greedy Best-First Search, A*, Heuristic Functions

1. Understanding of the Scenario

- A drone must fly from a depot to a destination and deliver medicine in a flood zone, quickly and safely.
- The environment is modelled as a **weighted graph**: nodes = waypoints/grid cells, edges = flight segments, **edge cost c(n,n′) = f(distance, terrain, weather, risk)**.
- The heuristic h(n) = straight-line (Euclidean) distance to the destination. Real-time updates about blocked paths are only **partial**, so the graph changes while the drone is flying.
- Goal: minimum delay and minimum risk → an informed search problem under dynamic, uncertain conditions.

2. Key Issues Identified

- **Dynamic environment:** roads/paths become blocked suddenly – a computed plan can become invalid.
- **Variable and non-uniform costs:** weather and terrain change travel cost, so shortest distance ≠ cheapest/safest path.
- **Heuristic is optimistic and crude:** straight-line distance ignores floods, wind and blocked areas.
- **Partial information:** only some blocked paths are reported.
- **Time-critical and safety-critical:** delays cost lives; a crash loses the medicine and the drone.

(a) Greedy Best-First Search (GBFS) in this scenario

GBFS expands the node that **appears closest to the goal**: **f(n) = h(n)**. It ignores the cost already spent g(n).

Behaviour: The drone keeps a priority queue ordered by straight-line distance and always flies toward the waypoint nearest the destination. In open terrain this produces a very fast, almost direct flight. If a flooded river or blocked road lies between the drone and the destination, GBFS still keeps choosing nodes that are geometrically nearest, so it may run into a dead end and then backtrack.

Strengths under uncertainty	Weaknesses under uncertainty
• Very fast decisions – expands few nodes (ideal when the drone must act in seconds). • Low memory and computation – suits on-board processors.	• **Not optimal** – may choose a path through severe weather because the goal looks “close”. • **Ignores g(n)** – cannot see terrain/weather cost or risk.

Strengths under uncertainty	Weaknesses under uncertainty
• Cheap to re-run after every update (quick replanning). • Works with only a heuristic and little knowledge of costs.	• **Can be incomplete** in graphs with loops or dead ends if no explored-set is kept. • Misled by the straight-line heuristic (flooded river, blocked road) – "goes the wrong way". • Worst-case time and space O(b^m).

(b) A* Search in this scenario

A* uses **f(n) = g(n) + h(n)** where g(n) = actual cost from the start to n (including weather and terrain penalties) and h(n) = estimated cost from n to the goal.

- **g(n)** pulls the search back from expensive, risky or long routes (reality / past cost).
- **h(n)** pushes the search toward the destination (estimate / future cost).
- The node with the lowest f(n) is expanded first, so A* balances "how much I have already paid" against "how much I probably still have to pay".

If h(n) is **admissible** (never overestimates: $h(n) \le h^*(n)$) and **consistent** ($h(n) \le c(n,n') + h(n')$), A* is **complete and optimal**. Straight-line distance is admissible if edge costs are scaled so that cost per unit distance is at least 1 (weather and terrain can only add cost, never reduce it below the straight-line distance).

Worked example (why A* is safer than Greedy)

Two routes from Start S to Goal G. Route 1 goes via B (a flooded bridge, storm = high cost). Route 2 goes via H (hill route, calm weather).

Node	Edge cost from parent	h(n)	g(n)	f = g + h
B (flood bridge)	S→B = 2	2	2	4
H (hill route)	S→H = 5	4	5	9
G via B	B→G = 12	0	14	14
G via H	H→G = 4	0	9	9

Greedy: compares h(B)=2 and h(H)=4, picks B, reaches G with total cost **14** (dangerous, expensive).
A*: expands B (f=4) and finds G with f=14, then expands H (f=9) and finds G with f=**9** – it returns the cheaper, safer route via H. A* therefore avoids the "greedy trap".

(c) Impact of heuristic accuracy

Heuristic quality	Effect on Greedy Best-First	Effect on A*
Accurate / close to true cost h*	Almost directly reaches the goal; very fast and often near-optimal.	Expands very few nodes; remains optimal.

Heuristic quality	Effect on Greedy Best-First	Effect on A*
Underestimates a lot (e.g., straight-line distance ignoring floods; h = 0 is the extreme)	Behaves poorly – gets attracted to blocked or flooded areas, wasted exploration.	Still **optimal** (admissible) but expands many more nodes; with h = 0 it becomes Dijkstra/Uniform-Cost Search (slow).
Overestimates (inadmissible)	No guarantee anyway – more aggressive, can become very wrong.	Faster, but **loses optimality** – may miss the cheapest/safest path (weighted A* trades quality for speed).
Inconsistent heuristic	Can cause erratic choices.	Nodes may be re-expanded, increasing cost; optimality needs re-opening nodes.

Key point: Greedy depends entirely on heuristic accuracy; A* depends on it only for **efficiency**, because g(n) keeps it correct. To improve the heuristic, the drone can use a weather-aware estimate, e.g. h′(n) = straight-line distance × minimum weather cost factor on the area (still admissible if the minimum factor is used).

(d) Effect of dynamic changes (blocked paths) on performance

- **Invalidated plans:** if a node/edge on the current path is blocked, the stored path and the g-values behind it become wrong; the drone must replan.
- **Greedy:** replanning is cheap and quick, but because it has no cost memory it may oscillate between two attractive nodes or re-enter a dead end.
- **Plain A*:** restarting from scratch after every change is expensive, as it repeats expansions and may miss the real-time deadline; but the new path found is still optimal for the current map.
- **Time-varying costs:** weather changes mean g-values go stale; the optimal path at time t may not be optimal at t+1.
- **Partial updates:** unknown blocked paths are assumed open (optimistic assumption), so the drone may discover a block late and have to detour.
- **Remedies:** replan from the drone's current position; use **incremental/anytime variants** – D* Lite, LPA*, Anytime Repairing A* (ARA*) – that reuse previous search effort and update only affected nodes; use a rolling planning horizon; keep an emergency landing/refuge node.

(e) Recommendation and justification

3. Decision: Use A* (with an admissible weather-aware heuristic) and an incremental replanning variant (D* Lite / weighted or anytime A*). Keep Greedy only as a fall-back.

- **Optimality / safety:** only A* uses g(n), so weather, terrain and risk costs are included. The path found is guaranteed minimum-cost (if h is admissible), so it avoids dangerous low-cost-looking routes.

- **Completeness:** A* with an explored set always finds a path if one exists, whereas Greedy can be trapped in dead ends.
- **Real-time constraints:** use weighted A* ($f = g + w \cdot h$, w slightly > 1) or anytime A* to get a good path quickly, then improve it while flying; D* Lite repairs the path after each blocked-path report instead of restarting.
- **Uncertainty:** replan from the current position each time new data arrives; include a risk penalty in edge costs so that unreliable areas are avoided.
- **Fall-back:** if computing time is almost zero (e.g., battery critical), Greedy can give an immediate heading, but this is a last resort.

Criterion	Greedy Best-First	A*
Evaluation function	$f = h$	$f = g + h$
Optimal?	No	Yes (admissible/consistent h)
Complete?	Not always	Yes
Speed	Fastest	Slower, but acceptable with weighted/anytime versions
Handles weather/terrain cost	No	Yes (through g)
Safety under dynamic change	Low	High (with D* Lite / replanning)

Question 2: Smart City Traffic Signal Optimization

Topic: Local Search and Optimization – Hill Climbing, Simulated Annealing, Genetic Algorithms

1. Understanding of the Scenario

- Hundreds of intersections must have their signal timings tuned together to minimise **waiting time, fuel consumption and congestion**.
- Here the **path to the solution is irrelevant** – only the final configuration (set of timings) matters. This is an **optimization problem**, ideal for local search.
- The search space is huge, traffic changes continuously, and the objective landscape has many local optima.

2. Key Issues Identified

- Extremely large (combinatorial) search space and coupling between neighbouring intersections.
- Multi-objective cost (waiting time, fuel, congestion).
- Dynamic, non-stationary traffic – the best solution keeps moving.
- Risk of getting stuck in local optima, plateaus and ridges.
- Each evaluation needs a traffic simulation/model, so evaluations are costly.

(a) Formulation as an optimization problem

Component	Definition for traffic signals
State (solution) x	Vector of signal parameters for N intersections: x = (cycle time, green split, offset) for each intersection i = 1…N, i.e. 3N decision variables.
Objective function	Minimise $F(x) = w1 \cdot \text{TotalWaitingTime}(x) + w2 \cdot \text{FuelConsumption}(x) + w3 \cdot \text{Congestion}(x)$. The weights reflect priorities (equivalent to maximising $-F(x)$ as the "elevation" in hill climbing).
Constraints	Minimum green time per phase, maximum cycle length, pedestrian crossing time, emergency vehicle priority, offsets for green-wave coordination.
Neighbourhood / moves	Change the green time of one phase by $\pm\Delta$, shift one offset, change one cycle time.
Evaluation	Traffic simulator or queueing model calculates F(x) from current flow data.

Why traditional (uninformed/informed tree) search is inefficient:

- **Huge search space:** if each intersection has only 10 × 10 × 10 = 1000 settings, 300 intersections give 1000^300 = 10^900 states – BFS/DFS/A* cannot enumerate or store this (exponential time and memory).
- **No explicit goal state or path:** there is no "goal test"; we want the best state, not a route to a known goal.
- **Dynamic problem:** by the time a systematic search finishes, traffic has changed and the answer is obsolete.
- Basic hill climbing stores the current state and a small amount of bookkeeping, so its memory use is typically O(1) with respect to the search depth. This claim does not apply to every local-search algorithm: genetic algorithms, for example, store a population of candidate solutions. Local-search methods can often return an available candidate before the computation ends.

(b) Hill Climbing: working and limitations

Hill climbing repeatedly moves to the best neighbouring state that improves the objective, and stops when no neighbour is better.

```
HILL-CLIMBING(problem):

 current <- initial timing plan (e.g., current field settings)

 loop:

   neighbor <- best successor of current   (one change in timing)

   if F(neighbor) >= F(current): return current   // local optimum

   current <- neighbor
```

Application: start from the present timing plan, try small changes (e.g., +2 s green on a busy road), simulate, keep the change if total waiting time falls, and repeat. Variants: steepest-ascent, first-choice, stochastic, and random-restart hill climbing.

Limitations: it gets stuck at local optima, plateaus and ridges; it never accepts a worse move, so it cannot escape; the result depends on the starting plan; it has no backtracking or memory; and it is sensitive to noisy traffic data and cannot track a moving optimum without restarting.

(c) Local maxima, plateaus and ridges – real-world interpretation

Problem	Meaning in the algorithm	Real-world traffic interpretation
Local maximum (local minimum of cost)	A state better than all neighbours but worse than the global best.	A corridor is tuned so that one junction is smooth, and any small change worsens the total delay; but a very different

Problem	Meaning in the algorithm	Real-world traffic interpretation
		coordination plan for the whole district would cut delay much more.
Plateau / shoulder	A flat region where neighbours have equal value, so there is no direction to move (shoulder = flat then rising).	Adding 1–2 s of green at a junction changes nothing because queues already clear (e.g., off-peak hours); the algorithm wanders or stops.
Ridge	A narrow high path whose direction is not along a single move; every single-variable move goes downhill.	A "green wave" needs the offsets of junction A **and** B changed together; changing either alone breaks the wave and increases waiting, so hill climbing rejects both moves.

(d) How Simulated Annealing and Genetic Algorithms overcome the limitations

Simulated Annealing (SA)

- Picks a **random** neighbour. If it is better, it is accepted; if worse ($\Delta E > 0$ in cost), it is accepted with probability $\mathbf{P = e^{(-\Delta E / T)}}$.
- Temperature T starts high (accepts many bad moves → exploration, escapes local optima and plateaus) and is lowered by a cooling schedule, e.g. $T \leftarrow \alpha T$ with $\alpha = 0.95$ (exploitation).
- With sufficiently slow cooling SA converges to the global optimum; it needs little memory and works for 3N variables; it can be "re-heated" slightly when traffic patterns change.
- Limitations: needs good tuning of the cooling schedule, and slow cooling is time-consuming.

```
SIMULATED-ANNEALING(problem, schedule):

 current <- initial plan

 for t = 1 to infinity:

   T <- schedule(t);  if T == 0: return current

   next <- random neighbor of current;   dE <- F(next) - F(current)

   if dE < 0: current <- next

   else: current <- next with probability e^(-dE/T)
```

Genetic Algorithm (GA)

- **Population** of candidate timing plans; chromosome = vector (cycle, split, offset for every intersection); fitness = 1 / F(x).

- **Selection** (fitter plans reproduce), **crossover** (e.g., combine the corridor-A settings of parent 1 with the district-B settings of parent 2 – this changes several variables together, which solves ridges), **mutation** (random small change, restores diversity), and **elitism** (keep best plans).
- Works in parallel with many solutions, so it explores the space globally; can be seeded with historical good plans and evaluated in parallel.
- Limitations: needs a population (more memory), each fitness evaluation requires a simulation, parameter tuning (population size, mutation rate), and slower convergence.

(e) Recommended approach and justification

3. Decision: Hybrid approach – Genetic Algorithm for global (periodic/offline) planning + Simulated Annealing (or hill climbing with random restarts) for continuous real-time fine-tuning, organised hierarchically by city zones.

- **Scalability:** divide the city into zones/corridors; each zone is optimised separately (decomposition), with limited coordination at the boundaries. The GA evaluates in parallel; SA needs memory for only one solution, so it scales to hundreds of intersections.
- **Adaptability:** GA creates good base plans for each time of day (morning peak, off-peak, events). SA starts from the current plan at a low-medium temperature and adapts in real time to changing flows; the temperature can be raised when a big change (accident, event) is detected.
- **Quality:** GA's crossover handles ridges; SA's probabilistic acceptance handles local optima and plateaus; plain hill climbing alone would be fast but gets stuck.
- **Practical safety:** every candidate must satisfy constraints (minimum green, pedestrian time); the system keeps the best-known plan as a fall-back and changes timings gradually.

Criterion	Hill Climbing	Simulated Annealing	Genetic Algorithm
Escapes local optima	No	Yes (probabilistic)	Yes (population + mutation)
Handles ridges	Poor	Moderate	Good (crossover)
Memory	Very low	Very low	High (population)
Speed per update	Very fast	Fast	Slower
Best role here	Quick local tweaks	Real-time adaptation	Global base-plan design

Question 3: Autonomous Rover Exploring Mars

Topic: Online Search Agent

1. Understanding of the Scenario

- A rover must reach sample sites on Mars in terrain it has **never seen**. It has limited sensors, so it sees only its surroundings.
- It has to interleave **sense → plan → act → learn** repeatedly: a real-time exploration problem.
- Wrong actions can be **irreversible** (falling into a crater, getting stuck in sand), so safety is as important as efficiency.

2. Key Issues Identified

- No complete map at the start; unknown action outcomes (state space unknown).
- Partial observability: limited sensing range, occlusion, noise.
- Dynamic environment: dust storms, shifting sand, changing lighting.
- Irreversible, hazardous actions; limited energy and time; communication delay with Earth (several minutes each way).
- Need to trade exploration against exploitation, and to keep a consistent internal model.

(a) Why an online search agent instead of offline search

Offline search	Online search
Computes the whole solution before acting, using a complete, known model of the world.	Interleaves computation and action: plan a little, act, observe, replan.
Needs the map, actions and step costs known in advance.	Learns the map by exploring; discovers actions and costs only after being in a state.
Plan can be tried in simulation, mistakes are free.	Every action is real; mistakes (irreversible actions) are costly.

- The Martian map is **unknown**, so an offline plan cannot be computed; the rover can only know a state's successors by actually visiting it (an exploration problem).
- Earth–Mars communication delay (roughly 4–24 minutes one way) makes remote real-time control impossible; the rover must decide **autonomously and locally**.
- The environment can change, so a pre-computed plan quickly becomes obsolete.
- Online agents are judged by the **competitive ratio** (cost actually travelled ÷ optimal cost if the map were known). Safe exploration requires that the state space is safely explorable (a goal remains reachable from every state).

(b) Challenges of partial observability and dynamic environments

Partial observability

- The rover sees only nearby cells; hazards behind rocks or over a ridge are **hidden** (occlusion).
- Sensor noise – shadows may look like craters (false positives), a crater edge may be missed (false negatives).
- Localisation error – wheel slip and drift make the rover unsure of its own position, so map entries can be wrong.
- The rover cannot distinguish a dead end from a long corridor without exploring, so it may need to backtrack.

Dynamic environment

- Dust storms reduce visibility and solar power; sand dunes shift, so previously safe cells become unsafe; lighting changes alter the perception of terrain.
- Old map information becomes stale; the model needs continual updating and "forgetting".

Resulting algorithmic problems

- Optimality is impossible to guarantee; the competitive ratio can be unbounded if dead ends and irreversible actions exist.
- Local-minima traps: a purely greedy-to-goal rover can get stuck in a U-shaped crater rim; there is a risk of loops (revisiting states).
- Resource limits: battery, thermal and computation constraints; the rover must replan cheaply.

(c) How the rover builds and updates its internal model

The rover keeps a **map (model) that starts empty** and grows with every observation:

- **Representation:** occupancy/cost grid (or graph) in which each cell is unknown, free or hazard (with a probability), plus terrain traversability (slope, roughness, sand). It also stores **result[s, a] = s′** (which action from which state led where), the visited states and the set of unexplored actions.
- **Sensing:** stereo cameras/LIDAR give obstacle information, IMU gives slope/tilt, wheel odometry gives movement; sensor data are fused (e.g., Bayesian/log-odds occupancy update) to reduce noise.
- **Localisation (SLAM):** the rover estimates its own pose and corrects its drift by matching landmarks (rocks, crater rims) so that new observations are written in the right cell.
- **Update:** after each move, newly seen cells change from unknown to free/hazard; costs and hazard probabilities are revised; outdated entries decay; new information triggers replanning.
- Learning heuristic values (LRTA*): the rover keeps H(s), an estimate of cost-to-goal, and revises it after each step. These updates can help the rover make progress through misleading local minima, but LRTA* does not guarantee safety or optimality in every environment.

```
ONLINE AGENT LOOP (sense - model - plan - act):

 repeat:
```

```
percept  <- sense()                    // local observation
model    <- UPDATE(model, percept, pose)  // map, result[s,a], hazards
if GOAL(s): stop
action   <- PLAN(model)   // e.g., A* on the known map / LRTA* / online DFS
if action is unsafe: choose safe alternative or backtrack
execute(action);  s <- new state


LRTA* update:  H(s) <- min over a of [ c(s,a,s2) + H(result[s,a]) ]
            move to the action a that gives that minimum
```

(d) Online search compared with uninformed and informed search

Feature	Uninformed (BFS, DFS, UCS)	Informed (Greedy, A*)	Online search (Online DFS, LRTA*, D* Lite)
Needs a known map?	Yes	Yes (plus a heuristic)	No – learns while moving
When does it act?	After planning completely	After planning completely	Interleaves planning and acting
Use of guidance	None (blind)	Heuristic h(n)	Heuristic learned/updated online
Backtracking	Simulated in memory (free)	Simulated in memory (free)	Physical backtracking – costs energy/time, may be impossible
Efficiency	Poor (exponential)	Good with a good heuristic	Good in practice; no optimal guarantee
Safety under unknown terrain	Not applicable	Not applicable (plan may be invalid)	Can be made safe with hazard-avoidance rules
Adapts to change	No	Only with replanning	Yes, naturally

Important: online search does not replace A*; the rover can run A* **on the part of the map it already knows** (a sliding horizon) and keep exploring the unknown part. Online DFS explores systematically

but needs physical backtracking and works only for reversible actions; LRTA* uses learnt heuristic values and escapes local minima.

(e) Most suitable approach and justification

3. Decision: A safety-aware online agent – frontier-based exploration + LRTA*/D* Lite (A* replanning on the partially known map) with a conservative hazard model.

- **Uncertainty:** the internal probabilistic map and sensor fusion give a representation of what is known and unknown; unknown cells are given a cautious cost.
- **Adaptability:** replans after each new observation (D* Lite reuses previous computations); LRTA* learns H(s) and cannot be trapped permanently by local minima.
- **Safety:** treat any cell with hazard probability above a threshold as an obstacle; keep safety margins around craters/rocks; prefer **reversible** moves; slow down in low-visibility areas; maintain a return-to-base/safe-haven path and a battery reserve; stop and re-sense if the model is inconsistent.
- **Efficiency:** goal-directed heuristic (distance to sample site) combined with exploring the frontier closest to the goal reduces wandering and energy use.
- **Autonomy:** all decisions are made on-board; Earth only supplies high-level goals.

Question 4: University Exam Timetable Generation

Topic: Constraint Satisfaction Problem – Backtracking, Constraint Propagation, Forward Checking

1. Understanding of the Scenario

- Assign every course exam a **time slot and exam hall** so that **all hard constraints** are satisfied (overlaps, capacity, precedence, faculty availability, accommodations).
- There is no objective to optimise first – the primary question is "find any assignment that satisfies all constraints" → a classical **CSP** (similar to graph colouring).
- Size grows quickly with the number of courses/students.

2. Key Issues Identified

- Many interacting constraints (binary, unary and global); possible over-constrained instances with no solution.
- Combinatorial explosion as courses, students and halls increase.
- Hard (must hold) vs soft (preferably hold) constraints – e.g., student fairness, gaps between exams.
- Special accommodations (extra time, accessible halls) restrict domains.
- Late changes (withdrawn courses, hall unavailable) require quick rescheduling.

(a) CSP formulation

Element	Definition
Variables (X)	One variable per course exam: **T_i** = time slot of exam i, and **H_i** = hall of exam i (or a combined variable E_i = (slot, hall)), for i = 1…n courses.
Domains (D)	D(T_i) = {1, 2, …, S} (e.g., 10 days × 2 sessions = 20 slots). D(H_i) = {halls with capacity ≥ enrolment of course i}. Unary constraints (faculty availability, accommodations) remove values from the domains in advance.
Constraint 1 – No student overlap	For every pair of courses (i, j) with at least one common student: **T_i ≠ T_j** (binary constraint).
Constraint 2 – Limited halls	Two exams cannot use the same hall in the same slot: if T_i = T_j then H_i ≠ H_j. In each slot, total candidates ≤ hall capacity (global/resource constraint, **AllDifferent** on (slot, hall) pairs).

Element	Definition
Constraint 3 – Precedence	If course i must be before course j: **T_i < T_j** (binary ordering constraint).
Constraint 4 – Faculty availability	Unary: T_i ∈ AvailableSlots(faculty of i). A faculty/invigilator also cannot be assigned to two exams in the same slot.
Constraint 5 – Special accommodations	Unary/binary: H_i ∈ accessible halls for such students; long-duration slots or separate rooms; not two exams of an accommodated student in the same day.

The **constraint graph** has courses as nodes and an edge when two courses share students or have precedence/resource conflicts; this is a graph-colouring problem where "colours" are time slots.

(b) How backtracking search works

Backtracking search is depth-first search that assigns **one variable at a time**, checks consistency with the constraints after each assignment, and **backtracks** (undoes the last assignment and tries the next value) when no legal value remains. Only one variable is considered per level, so the tree has depth n and d^n leaves in the worst case (assignments are commutative).

```
BACKTRACK(assignment, csp):
 if assignment is complete: return assignment
 var <- SELECT-UNASSIGNED-VARIABLE(csp)        // MRV, degree heuristic
 for value in ORDER-DOMAIN-VALUES(var, csp):   // least-constraining value
   if value consistent with assignment:
     add {var = value};  (inference: forward checking / AC-3)
     result <- BACKTRACK(assignment, csp)
     if result != failure: return result
     remove {var = value}                      // backtrack
 return failure
```

In the timetable: choose an exam (e.g., the course with the most conflicts), try slot 1 and a hall; check that no common-student course already has slot 1, the hall is free, faculty is available and precedence holds; if OK, go to the next course; if a later course has no valid slot/hall, undo the previous choice and try the next slot. **Improvements:** MRV (choose the variable with the fewest legal values), degree

heuristic (most constraints with unassigned variables), LCV (choose the value that rules out the fewest options).

(c) How constraint propagation (forward checking) improves efficiency

Forward checking (FC): whenever a variable X is assigned, delete from the domains of its unassigned neighbours all values inconsistent with that assignment. If any domain becomes **empty**, the algorithm backtracks immediately instead of discovering the failure much deeper in the tree.

Mini-example
Courses A, B, C, D; slots {1, 2, 3}. A, B, C share students with each other (all pairwise conflicts); D must be after A.

Step	Action	Domain of A	B	C	D
0	Initial domains	{1,2,3}	{1,2,3}	{1,2,3}	{1,2,3}
1	Assign A = 1 → forward check	1	{2,3}	{2,3}	{2,3} (D > A removes 1)
2	Assign B = 2 → forward check	1	2	{3}	{2,3}
3	Assign C = 3 → D	1	2	3	{2,3} → D = 2 or 3 ✓

For illustration, suppose an additional course E conflicts with A, B and C, and all four courses must be assigned different time slots. If only three slots are available, assigning A, B and C to the three different slots leaves E with no legal value. Forward checking detects E's empty domain and backtracks immediately. This example demonstrates early failure detection; the actual result depends on the specified conflict graph and domains.

- **Benefits:** earlier detection of dead ends (reduces thrashing), smaller domains for later choices, and the pruned domains make **MRV** more effective.
- **Limitation:** FC checks only constraints between the assigned variable and unassigned ones; it does not detect conflicts between two unassigned variables. Stronger propagation – **arc consistency (AC-3 / MAC, maintaining arc consistency)** – or global-constraint propagation (AllDifferent for halls) detects more failures at higher cost per node.
- **Further improvements:** conflict-directed backjumping, nogood learning.

(d) Real-world challenges and the best strategy for scalability

Challenge	Consequence	Handling
Very large size (hundreds of courses, thousands of students)	Plain backtracking explodes (d^n).	MRV + degree + LCV, forward checking/MAC, decomposition into clusters (departments/semesters), restarts.

Challenge	Consequence	Handling
Over-constrained instances (no solution)	Search fails or runs indefinitely.	Separate hard and soft constraints; relax or prioritise constraints; report conflicting constraints to the administrator.
Special accommodations and faculty limits	Very small domains, tight interaction.	Apply unary constraints first (node consistency); schedule highly constrained exams first (MRV).
Last-minute changes	Complete re-solving wastes time and disturbs students.	Local repair with **min-conflicts** local search starting from the existing timetable.
Soft-quality aspects (fairness, gaps between exams)	CSP solution may be valid but poor.	Post-optimise with local search/simulated annealing or a weighted (constraint optimisation) cost.

3. Decision: A hybrid strategy – preprocess (node/arc consistency), then backtracking with MRV + degree + LCV and forward checking/MAC; use a graph-colouring ordering (e.g., largest-degree/DSATUR) to guide the search; decompose by department/cluster; and use min-conflicts local search for repair and soft-constraint improvement.

- **Why:** systematic backtracking with propagation is complete and guarantees that every hard constraint is met; heuristics and propagation reduce the exponential search drastically; decomposition and local repair make it scalable and responsive to changes.
- **Result:** valid, conflict-free timetables in reasonable time, with the ability to adapt when data changes.

Question 5: Strategic Game AI for Real-Time Decision Making

Topic: Adversarial Search – Minimax, Alpha-Beta Pruning, Evaluation Functions

1. Understanding of the Scenario

- An AI opponent in a real-time strategy (RTS) game competes against a human; it must pick moves within strict time limits in a game with a very large state space.
- This is a **competitive, two-player (adversarial)** situation. Classical tool: **Minimax** with **Alpha-Beta pruning** and an **evaluation function** for depth-limited search.

2. Key Issues Identified

- Enormous branching factor and depth – exponential game tree.
- Strict time limit – full search impossible; anytime/cut-off search required.
- Complex state evaluation (economy, army, position) – need a good heuristic.
- RTS specifics: real-time/simultaneous actions, partial information (fog of war) and some randomness – need abstractions.

(a) How Minimax models the problem

Minimax treats the game as a **two-player, zero-sum, turn-based** game between **MAX** (the AI) and **MIN** (the human, assumed to play optimally against the AI). The game is defined by: initial state, PLAYER(s), ACTIONS(s), RESULT(s, a), TERMINAL-TEST(s) and UTILITY(s).

MINIMAX(s) = UTILITY(s) if s is terminal; max over actions a of MINIMAX(RESULT(s,a)) if PLAYER(s) = MAX; min over actions a of MINIMAX(RESULT(s,a)) if PLAYER(s) = MIN.

- MAX picks the move that **maximises** the value, assuming MIN will reply with the move that **minimises** it – this gives the best guaranteed outcome against a rational opponent.
- For an RTS, time is discretised into decision steps (or macro-actions such as “build barracks”, “attack base”); nodes are game states (units, resources, positions), edges are action choices; values are propagated from the leaves (or depth cut-off with an evaluation function) to the root.

```
MINIMAX-DECISION(state): return argmax over a of MIN-VALUE(RESULT(state,a))

MAX-VALUE(s): if CUTOFF(s) return EVAL(s); v <- -inf

  for a in ACTIONS(s): v <- max(v, MIN-VALUE(RESULT(s,a)))

  return v

MIN-VALUE(s): if CUTOFF(s) return EVAL(s); v <- +inf

  for a in ACTIONS(s): v <- min(v, MAX-VALUE(RESULT(s,a)))

  return v
```

Example: root (MAX) has three MIN children with leaves B = (3, 12, 8), C = (2, 4, 6), D = (14, 5, 2). MIN values: B = 3, C = 2, D = 2. MAX chooses **3** (move to B).

(b) Computational challenges of large game trees

- **Time complexity O(b^d)** and space O(b·d) (depth-first): with branching factor b and depth d, the number of nodes explodes. Chess has b ≈ 35 and d ≈ 80, i.e. about 35^80 ≈ 10^123 nodes – impossible. An RTS game is worse: many units can each choose among many actions, so b can be hundreds or thousands.
- **Time limit:** the AI must answer in milliseconds to seconds, so only a small part of the tree can be explored; the search must be cut off at a depth limit and use an evaluation function.
- **Evaluation cost:** evaluating each state is expensive (many units, map, resources), further reducing the number of nodes per second.
- **Horizon effect:** cutting off at a fixed depth can hide an inevitable loss or gain just beyond the horizon; mitigated by quiescence search and iterative deepening.
- **Real-time complications:** continuous time, simultaneous moves, hidden information (fog of war) and randomness violate the perfect-information turn-based assumptions of plain minimax, so abstraction is required.
- **Repeated states:** the same position can be reached by different move orders; without a transposition table the work is duplicated.

(c) How Alpha-Beta pruning improves efficiency

Alpha-Beta computes **exactly the same minimax decision** but **prunes branches that cannot influence the final choice**. It carries two bounds along the path:

- **α (alpha)** = the best (highest) value that MAX can already guarantee on the path to the root (a lower bound).
- **β (beta)** = the best (lowest) value that MIN can already guarantee on the path to the root (an upper bound).
- **Pruning rule:** stop exploring the remaining children of a node as soon as **α ≥ β**. Reason: at a MIN node, once a child has value ≤ α, MIN can force a result no better than what MAX already has elsewhere, so MAX will never choose this branch; at a MAX node, once the value ≥ β, MIN above would never allow this branch.

```
MAX-VALUE(s, alpha, beta):

 if CUTOFF(s): return EVAL(s)

 v <- -inf

 for a in ACTIONS(s):

   v <- max(v, MIN-VALUE(RESULT(s,a), alpha, beta))
```

```
    if v >= beta: return v        // prune remaining children
    alpha <- max(alpha, v)
  return v


MIN-VALUE(s, alpha, beta):
  if CUTOFF(s): return EVAL(s)
  v <- +inf
  for a in ACTIONS(s):
    v <- min(v, MAX-VALUE(RESULT(s,a), alpha, beta))
    if v <= alpha: return v       // prune remaining children
    beta <- min(beta, v)
  return v
```

Detailed reasoning with the same example

- Explore B = (3, 12, 8): B = 3, so at the root $\alpha = 3$.
- Explore C: first leaf = 2. C is a MIN node, so $C \leq 2 < \alpha = 3$. MAX already has 3 from B, so C can never be chosen → **leaves 4 and 6 are pruned**.
- Explore D: leaves 14, 5, 2 give D = 2 (14 and 5 do not allow pruning because they are above α, and the final 2 gives D = 2 < 3). Root value = max(3, 2, 2) = **3** – identical to minimax, with two fewer leaves examined.

Case	Nodes examined	Comment
Worst case (bad move ordering)	O(b^d)	No pruning; same as minimax.
Random/average ordering	about O(b^(3d/4))	Moderate improvement.
Best case (perfect ordering)	O(b^(d/2))	Square root of the nodes: the **search depth can be doubled** in the same time (e.g., b = 35: effective branching ≈ 6).

Move-ordering heuristics bring practice close to the best case: try captures/attacks and previously best ("killer", history heuristic) moves first, use the best move from the previous iterative-deepening iteration, and use transposition tables to avoid repeating states.

(d) Design of an evaluation function

When the search is cut off, a heuristic Eval(s) estimates the utility of a non-terminal state and should correlate with the chance of winning while remaining fast to compute. Terminal states should receive values that dominate all non-terminal evaluations; practical implementations usually use sufficiently large finite values (often adjusted for win/loss distance) rather than literal infinity. A weighted linear form is used:

Eval(s) = w1·f1(s) + w2·f2(s) + … + wk·fk(s) (features are normalised to the same scale and are difference values: AI − opponent)

Feature f_i	What it measures	Why it matters	Sample weight
f1: Military strength	Total value of army (unit cost × health, unit counters) AI − enemy	Armies decide battles directly	0.30
f2: Economy	Resources, income rate, number of workers/bases	Future production power	0.20
f3: Technology / upgrades	Tech level, upgrades researched, special units available	Long-term advantage	0.10
f4: Map and territory control	Controlled resource points, vision/scouting, chokepoints	Safe economy and attack options	0.15
f5: Base safety / threat	Base health, defenders, distance of enemy forces to key buildings	Prevents a game-losing attack	0.15
f6: Positional / mobility	Unit formation, high ground, number of available actions	Flexibility to attack or defend	0.10

Example: if f1 = +0.4 (stronger army), f2 = −0.2, f3 = 0, f4 = +0.3, f5 = +0.1, f6 = 0, then Eval = 0.30(0.4) + 0.20(−0.2) + 0 + 0.15(0.3) + 0.15(0.1) + 0 = 0.12 − 0.04 + 0.045 + 0.015 = **0.14** → slightly favourable to the AI.

Design justification: a combination of material (military), economy and position reflects how expert players judge a position; weights are tuned from expert knowledge, then refined by **self-play or machine learning (e.g., temporal-difference learning, regression on game outcomes)**; features must be cheap so that more nodes can be searched; use quiescence search so that evaluation is applied only to "quiet" states (not in the middle of a battle).

(e) Strategy for balancing optimality and real-time performance

3. Decision: Depth-limited Alpha-Beta search with iterative deepening under a time budget, strong move ordering, transposition tables and a good evaluation function; use abstraction/selective search (and Monte-Carlo Tree Search where the branching factor is too large) for the real-time setting.

- **Iterative deepening (anytime):** search depth 1, then 2, 3 … until the time limit; always keep the best move of the last completed depth. The AI always has a legal, good move, and the earlier iterations improve move ordering for later ones (little overhead).
- **Alpha-Beta + move ordering + transposition table:** nearly halves the effective depth exponent without losing the minimax optimum for the searched depth.
- **Depth cut-off + evaluation function + quiescence search:** limits computation, avoids the horizon effect.
- **Reduce the branching factor:** use high-level macro-actions (strategy level: build/attack/expand; tactic level: unit micro), forward pruning (beam search, discard clearly bad moves), and opening books/learned policies.
- **Handle RTS specifics:** treat unknown parts of the map with probabilistic estimates or sampling; use Monte-Carlo Tree Search (UCT) when branching is too large or information is hidden.
- **Trade-off control:** adapt depth and time per move to the situation (more time for critical battles), and difficulty settings can reduce depth to make the AI beatable.

Technique	Optimality	Speed	Role
Full Minimax	Optimal for the whole game	Impractically slow	Conceptual model only
Depth-limited Minimax + Eval	Optimal only w.r.t. Eval	Still slow	Basic version
Alpha-Beta (ordered) + Iterative deepening	Same result as Minimax at given depth	Up to ~2× deeper search	**Core engine (recommended)**
MCTS / abstraction	Approximate (improves with time)	Anytime; scales to huge b	Complement for very large or uncertain parts